

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/362345190>

DESIGN AND IMPLEMENTATION OF SMART GLASSES FOR BLIND PEOPLE – Using Raspberry PI

Thesis · January 2022

DOI: 10.13140/RG.2.2.11626.59844

CITATIONS

3

READS

4,906

1 author:



[Rotimi Abayomi](#)

Nile University of Nigeria

1 PUBLICATION 3 CITATIONS

SEE PROFILE

**DESIGN AND IMPLEMENTATION OF SMART
GLASSES FOR BLIND PEOPLE**

BY

**ROTIMI ABAYOMI
171203008**

**A FINAL YEAR REPORT SUBMITTED TO THE DEPARTMENT
OF COMPUTER ENGINEERING**

**IN PARTIAL FULFILMENT FOR THE AWARD OF THE DEGREE
OF BACHELOR OF ENGINEERING (B.ENG.) IN COMPUTER
ENGINEERING**

**FACULTY OF ENGINEERING
NILE UNIVERSITY OF NIGERIA, ABUJA**

JUNE, 2022

APPROVAL

This project work titled “**Design and Implementation of Smart Glasses for Blind People**” by **Rotimi Abayomi** with ID Number: **171203008** satisfies all the requirements necessary for the award of the degree of Bachelor of Engineering (B.Eng.) in Computer Engineering, and has been found credible for its contribution to knowledge and its practical approach for future implementation of the system.

Dr. Ali Nyangwarimam Obadiah
Supervisor

Assoc. Prof. Sadiq Thomas
Head of Department (HOD)

Prof. Steve Adeshina
Dean of Faculty

Prof. Danjuma D. Dajab
External Supervisor

DECLARATION

I, **ROTIMI ABAYOMI**, hereby declare that this project titled “**Design and Implementation of Smart Glasses for Blind People**” has been carried out by me under the supervision of Dr Ali Nyangwarimam Obadiah, Department of Computer Engineering. It is presented for the award of the Degree of Bachelor of Engineering (B.Eng) in Computer Engineering. All the sources of information that are externally obtained are acknowledged by means of reference.

Signature

Date

DEDICATION

This project is dedicated to my loving father, Mr. Abayomi Folorunsho, and my supportive mother, who have always been there for me in my pursuit of knowledge. Without your constant love and support, none of this would have been possible.

ACKNOWLEDGEMENT

First and foremost, I want to thank Jehovah for allowing this effort to take place.

I'd also want to thank my family for their unwavering support and compassion during my stay at university.

Dr. Sadiq Thomas, thank you for all of your hard work and devotion as the Head of the Department of Computer Engineering. My lecturer, Dr. Ali Obadiah, deserves special thanks for his time, patience, and invaluable support. I'd want to express my gratitude to Engr. Precious for his support throughout the project's timeline.

I'd want to express my gratitude to my Nile University classmates for all we learned together throughout this project.

ABSTRACT

Even with the aid of canes or sticks, blind people have a difficult time walking on their own. They must always be accompanied to regions where they must go, whether by trained dogs or people; nonetheless, there may be times when there is no supervisor present and they are stranded due to mobility limits. This difficulty is addressed by this smart glass. Its approach is based on object identification and depth estimation. This approach examines the environment for potential threats along a walk path, including the distance to the object and the type of object, and notifies the user of the danger, allowing the user to move around freely without continual monitoring. After the model detects the object, an algorithm tracks the distance to the selected user and provides the user with the necessary information required to process his environment. The glass's camera collects data from the user's surroundings and delivers it to the model, which scans it and gives input on probable directions. This smart glass is a tremendous help to blind individuals who are always frustrated by the fact that, due to their condition, they must be constantly overseen by others, relieving their anxieties a little by providing new solutions to their problems.

TABLE OF CONTENTS

APPROVAL	I
DECLARATION	II
DEDICATION	III
ABSTRACT	V
LIST OF FIGURES	IX
LIST OF TABLES	X
CHAPTER ONE.....	1
INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement.....	2
1.3 Aim and Objectives.....	3
1.4 Significance of the Study	3
1.5 Scope of Study	3
1.6 Limitations of the Study	3
CHAPTER TWO.....	5
LITERATURE REVIEW	5
2.1 Introduction.....	5
2.2 Fundamental Theory.....	5
2.2.1 Computer vision	5
2.2.2 Stereo vision and camera.....	6
2.2.3 Object Detection.....	7
2.3 Review of past works.....	8
2.3.1 Smart glasses for blind people.	8
2.3.1 Low Cost Ultrasonic Smart Glasses for Blind.	8
2.3.2 Autonomous Walking Stick for the Blind Using Echolocation and Image Processing.....	9

2.3.3 Smart Glasses using deep Learning and Stereo Camera.....	10
2.3.4 Planning the Path and Avoidance Obstacles for Visually Impaired/Blind People.	11
2.3.5 Review of Past Works: Summary	12
2.4 Main components of the system	14
2.4.1 Camera module.....	14
2.4.2 Raspberry pi 4b.....	15
CHAPTER THREE	16
RESEARCH DESIGN AND METHODOLOGY	16
3.1 Introduction.....	16
3.2 Hardware design	16
3.2.1 Block diagram	16
3.2.2 Headphones.....	17
3.2.3 Raspberry pi 4 b.....	18
3.2.4 Vibration motor.....	18
3.2.5 Design specifications	19
3.2.6 Power Draft.....	20
3.3 Software design	20
3.3.1 Raspbian OS.....	21
3.3.2 Python Programming	22
CHAPTER FOUR	28
IMPLEMENTATION AND RESULT.....	28
4.1 Introduction.....	28
4.2 Implementation	28
4.3 Mode of Operation	32
4.4 Results	33
4.5 Bill of Engineering Measurements And Evaluation.....	36
CHAPTER FIVE	37
CONCLUSION AND RECOMMENDATION	37

5.1 Conclusion.....	37
5.2 Challenges Encountered	37
5.3 Recommendation.....	38
5.4 Summary	38
REFERENCES	39
APPENDIX	41

LIST OF FIGURES

Figure 1. 1 Blind Person walking [1].....	2
Figure 2. 1 Raspberry Pi Camera Rev2.0 5MP [1]	6
Figure 2. 2 Object Detection [2]	7
Figure 3. 1 Block diagram of the proposed system	16
Figure 3. 2 Earpiece [3].....	17
Figure 3. 3 Raspberry pi 4b [4].....	18
Figure 3. 4 Vibration motor [5].....	19
Figure 3. 5 Flowchart of proposed system	21
Figure 3. 6 Chess Board Pattern.....	24
Figure 3. 7 Chess Board Pattern Identified	24
Figure 4. 1 Connection between Raspberry pi and Glasses.....	29
Figure 4. 2 Close-up to 8MP (USB) Camera's	29
Figure 4. 3 Close-up to Vibration motor	30
Figure 4. 4 Glasses open to be worn by user.....	31
Figure 4. 5 Triangulation Stereo Vision	32
Figure 4. 6 Camera threads running	33
Figure 4. 7 Object detected.....	34
Figure 4. 8 GTTS audio function running and depth printed.....	34

LIST OF TABLES

Table 2. 1 Review of past works	12
Table 3. 1 Raspberry pi 4b design specification	19
Table 3. 2 USB Camera Module design specification	20
Table 4. 1 Bill of materials.....	36

CHAPTER ONE

INTRODUCTION

1.1 Background of Study

For decades, blindness has been a concern for many people all around the world. According to WHO, there are at least 2.2 billion people worldwide who have near or distance vision impairment [1]. From a poll also conducted in Nigeria, 1.13 million people under the age of 40 are blind [2]. As a distinct group of people in society, there has always been a desire to provide the finest possible living conditions. When traveling outside, this is especially crucial because safe navigation is required as seen in Fig 1.1 of a blind individual walking outside. Some of the problems faced during outdoor navigation are falling from stairs, falling into holes or pits, bumping into objects in footpaths, falling onto underground rails, falling into gaps between buses and their platforms and bumping into building scaffolds and glass doors. Throughout the years, a variety of safe navigation methods have been introduced, ranging from human assistance to more technology ways. Despite the fact that technological advancements have given these individuals an advantage, outside navigation can still be a frightening experience. Due to this necessity, there is a need to implement a safer navigation system.

In an attempt to tackle this problem, a lot of research has been made to implement a good and safe navigation system. One of the methods implemented is the use of stereo vision (computer vision) to monitor a 3D space. This technology recognizes and alerts users/blinds to obstructions in their route. This will highly benefit the user who may not be able to see the obstacles ahead.



Figure 1. 1 Blind Person walking [3]

1.2 Problem Statement

One of the most misunderstood disabilities is blindness. The general populace has preconceptions about the blind that they feel are accurate. Some people feel that the blind can't work or live a regular life on their own. This contrasting cultural perspective and reality of a blind person's life has been depicted in numerous literatures[4]. There is some truth in the public's perception, as the blind do experience difficulties.

Some of these challenges include being unable to move, requiring assistance from others, and so on. It can be difficult to perceive an obstacle or detect it in time. Most of the time, individuals must become acquainted with their surroundings in the hopes that nothing will change. This is a possible danger since it could result in serious injuries or even death.

Being able to assist this blind person in understanding and identifying potential hazards can help them avoid harm. Nothing is more essential than preserving a life; in this situation, we are assisting them in living their own life.

1.3 Aim and Objectives

The aim of this project is to design and construct a Smart glass for blind people with the following objectives:

- To design a Smart glass using Raspberry pi
- To implement the Smart glass using Raspberry pi
- To test and evaluate the performance of the glass.

1.4 Significance of the Study

According to the WHO, 2.2 billion people are blind; therefore assisting them will help them integrate into society. Giving blind people a way to become 1% more self-sufficient will benefit them, their caretakers, and society as a whole. The proposed technology will assist these people in staying safe. Avoiding avoidable injuries and assisting them in moving about more easily.

1.5 Scope of Study

For this scope, the system employs an object recognition system that recognizes just individuals and vehicles. This smart glass's complexity is anticipated to supply its user with a plethora of data to comprehend the environment and move accordingly.

1.6 Limitations of the Study

Despite the many benefits of the proposed system, it does not address the drawbacks of others. Obstacles such as flying objects, staircases, and potholes provide a challenge when navigating with this equipment.

Although it aims to replace the human eye for the blind, this project will never be as excellent as the human eye. In addition, the Raspberry Pi's computing capacity is limited. A board with a more powerful GPU or TPU is required to handle the computations.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter will include the evaluation of prior projects by other scholars, as well as the primary concept of the project, the explanation of numerous units, and general principles.

2.2 Fundamental Theory

This section is focused more on the fundamental concepts related to the Smart glasses for blind people.

2.2.1 Computer vision

Computer vision is a branch of artificial intelligence that leverages data from films, photographs, and other visual inputs to assist systems in making judgments based on that data. In simple words, computer vision helps systems to understand what they see. [5]

Human vision is similar to computer vision, however humans have an advantage. Human vision benefits from lifetimes of context to teach it how to distinguish objects apart, how far away they are and if they are moving. [5]

In order to execute computer vision, a system must be taught. It must employ cameras, data, and algorithms, unlike the human sight and brain. Hundreds of processes can be handled in minutes in some instances due to the quality of the model learned and its system. In terms of speed, this can almost match human capabilities, but nothing near its efficiency. [5]

2.2.2 Stereo vision and camera

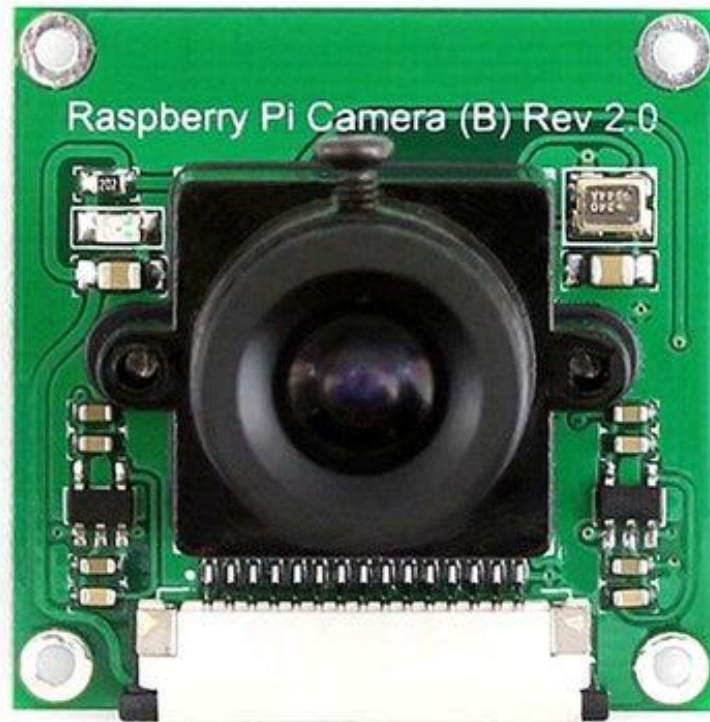


Figure 2. 1 Raspberry Pi Camera Rev2.0 5MP [6]

Stereo vision is a subset of computer vision that operates on the same principles as computer vision but with slight variations. A stereo camera is one that has two or more image sensors. This allows the camera as shown in fig 2.1 to mimic human binocular vision. This vision gives it the capacity to discern depth.

The calculation of depth based on the binocular disparity between the pictures of an object in the left and right eyes is known as stereo vision.

2.2.3 Object Detection



7

2.3 Review of past works

2.3.1 Smart glasses for blind people.

This project designed and constructed smart glasses to help blind people read. These glasses are also used to translate typed text created in English. Any text picture is scanned and converted into auditory text by the glasses. The audio will be listened to through a headphone that is attached to the glasses. To accomplish the tasks, the glasses employed OCR, (gTTS), and Google translation. The Ultrasonic sensor in the glasses is used to measure the needed distance between the user and the photo item in order to snap a clean picture.[9]

This version's smart glasses technology can only be used to read text. Students, researchers, and those who enjoy reading in print are the target audience. Although, in today's world, many texts are now available in audio format, allowing users to listen to text at a faster rate than before. Various technologies have superseded it, lowering its worth.

2.3.1 Low Cost Ultrasonic Smart Glasses for Blind.

This project constructs a smart glass that uses the ultrasonic sensor for detection. This project focuses on navigating, which is one of the most crucial attributes for the blind. This method is very inexpensive, saves time in construction, and is light and portable. The project's components are easily available, which speeds up building. The initiative, as excellent as it appears to be and as simple as it appears, is not without flaws.[10]

The ultrasonic sensor utilized in this approach has a number of flaws. This sensor is unable to effectively portray and anticipate the user's environment. It takes only one step ahead. Occasionally, multiple of

these sensors must be gathered. The sensor has a restricted range to go along with this. It is impossible to properly analyze the surroundings in order to provide an appropriate user route. Furthermore, only one ultrasonic sensor was employed in the centre of the glasses in this project. As a result, only forward movement may be detected.

2.3.2 Autonomous Walking Stick for the Blind Using Echolocation and Image Processing.

The goal of this project is to develop and build a smart walking stick that will aid visually impaired persons in identifying obstacles and getting to their destination. The Assistor was the name given to this construction. The Assistor relies on echolocation, image processing, and a navigation mechanism to function. The Assistor detects things using ultrasonic sensors that echo sound waves. The model has a Smartphone app which is used to lead the user to their destination using GPS (Global Positioning System) and maps, and an image sensor is used to identify the items in front of the user and for navigation by capturing runtime photographs.[11]

This project, like the last one, makes use of an ultrasonic sensor. It's complemented with a GPS navigation system this time. The navigation system has been pre-programmed to assist the user in getting to their preferred place. For each circumstance, the user receives auditory feedback. Image matching is also included in the system. Using algorithms, determine the best route to the destination, to assist the user in getting to their goal.

A disadvantage to this is that picture processing is not real-time. The photos taken must be processed once they have been captured. This

necessitates a significant amount of computational power and time. Before the picture has finished processing, the risk has happened. The model compensates for this flaw by including an ultrasonic sensor. It's possible that the solution won't work in every instance.[11]

2.3.3 Smart Glasses using deep Learning and Stereo Camera.

This project uses Deep Learning and Stereo Vision to design and build a Smart Glass. The distance between the blind and the obstacle is calculated using stereo cameras, and a gyro sensor is used to make a distinction between the floor and obstructions. The vibration motor and buzzer are located on the front, left, and right sides, respectively. The vibration motor and buzzer respond to the distance between the obstacle and the user. The right vibration motor and buzzer activate when an impediment approaches from the right side. When approaching from the left side, however, the left vibration motor and buzzer are activated. The front vibration motor and buzzer activate when an impediment approaches from the front. A deep learning technique is utilized to recognize the type of barrier once it has been detected. As a deep learning approach, this model employed YOLO v3 (You Only Look Once version 3).[12]

The model operates similarly to human eyes, making it an excellent fit for assisting the blind. The model, however, does not provide the path to a specific location. With all of this, its structure and intricacy offer it an advantage over the competition.

2.3.4 Planning the Path and Avoidance Obstacles for Visually Impaired/Blind People.

The A* algorithm is used in this design to compute a path from the user to the target location. A* is one of several search algorithms that takes an input and examines a number of different pathways before returning a result. A* (as A star) is a computer method frequently used in route finding and graph traversal, the act of charting an efficiently traversable path between points called nodes in computer science. This design considers the start location, destination point, and obstacles, then implements the algorithm and generates an appropriate obstacle avoidance path navigation.[13]

Pathfinder algorithms like A* allow you to plan ahead of time rather than waiting until the problem is discovered. Rather than reacting to an issue, they take proactive measures. It has the drawback of being slower than the other algorithms.

Despite the fact that this design does not explicitly implement a model or concept to aid the blind, it is nonetheless useful. This concept shows how a navigation system should work.

2.3.5 Review of Past Works: Summary

Table 2. 1 Review of past works

TITLE	IDEA	DRAWBACK	BENEFIT	IMPROVEMENT
Smart glasses for blind people [9]	Translate typed text to English	Not applicable in modern day		Wider range of uses
Low Cost Ultrasonic Smart Glasses For Blind [10]	Using Ultrasonic sensors for detection	Limited range	Cheap	There are other sensors better than the ultrasonic
Autonomous Walking Stick For The Blind Using Echolocation And Image Processing [11]	Using image processing to recalculate path Using GPS	Too Complex and slow	Image processing	Reduce complexity Make image processing realtime

Smart Glasses using Deep Learning and Stereo Camera [12]	Using stereo vision for 3D depth estimation and object detection	No path planning implemented	Accurate depiction of environment Object detection	Path planning algorithm implemented
Planning the Path and Avoidance Obstacles for Visually Impaired/Blind People [13]	Design of A* path planning algorithm	No implemented solution or hardware component involved	Path planning algorithm implemented	Implement a hardware solution to problem

2.4 Main components of the system

2.4.1 Camera module

This is one of the main components of the system. This camera module sends live feed to the raspberry pi for required computations. The camera module chosen for this case is the 8MP 1080P USB Camera Module.

Features

- Sensor: 8MP 1/4" IMX219
- Resolution: 8MP 3264H x 2448V
- Data Format: MJPG/YUY2
- Frame Rates:
 - MJPG 15fps @ 3264x2448, 30fps @ 1920x1080
 - YUY2 30fps @ 640x 480.
- Data Format: MJPG/YUY2
- Field of View (FOV) : H=62.2°, V=48.8°
- IR Sensitivity: Integral IR filter, 650nm IR Filter, visible light only
- Adjustable Features: Brightness, Contrast, Saturation, Sharpness, Gamma, White balance.
- Power Supply: USB Powered 5V
- Working Current: MAX 200mA
- Operating Temp.: -4°F~167°F (-20°C~+75°C)
- Dimension: 38mm x 38mm

2.4.2 Raspberry pi 4b

In terms of computing power, the Raspberry Pi 4 ships with a new 1.5GHz Quad-core CPU.

Backing up that processing power is *up to* a whopping 8GB of *faster* LPDDR4 RAM – the Raspberry Pi 4 comes in 1GB, 2GB, 4GB and 8GB options.

With this new level of hardware, the Raspberry Pi 4 truly is a viable dual-display desktop computer.

Technical Specifications:

- 1.5GHz 64-bit quad-core CPU
- Available with 1GB, 2GB, 4GB or 8GB RAM
- 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless, Bluetooth 5.0, BLE
- Gigabit Ethernet
- 2x USB 3.0 ports; 2x USB 2.0 ports.
- Raspberry Pi standard 40 pin GPIO header
- 2 × micro-HDMI ports (up to 4kp60 supported)
- 2-lane MIPI DSI display port
- 2-lane MIPI CSI camera port
- 4-pole stereo audio and composite video port
- 265 (4kp60 decode), H264 (1080p60 decode, 1080p30 encode)
- OpenGL ES 3.0 graphics
- Micro-SD card slot for loading operating system and data storage
- 5V DC via USB-C connector (minimum 3A*)
- 5V DC via GPIO header (minimum 3A*)

CHAPTER THREE

RESEARCH DESIGN AND METHODOLOGY

3.1 Introduction

The hardware and software in the proposed system are the two essential components. The hardware and software components used are described in detail in this section.

3.2 Hardware design

The various sub-components of the hardware system that have been used to make the smart glasses system are described below.

3.2.1 Block diagram

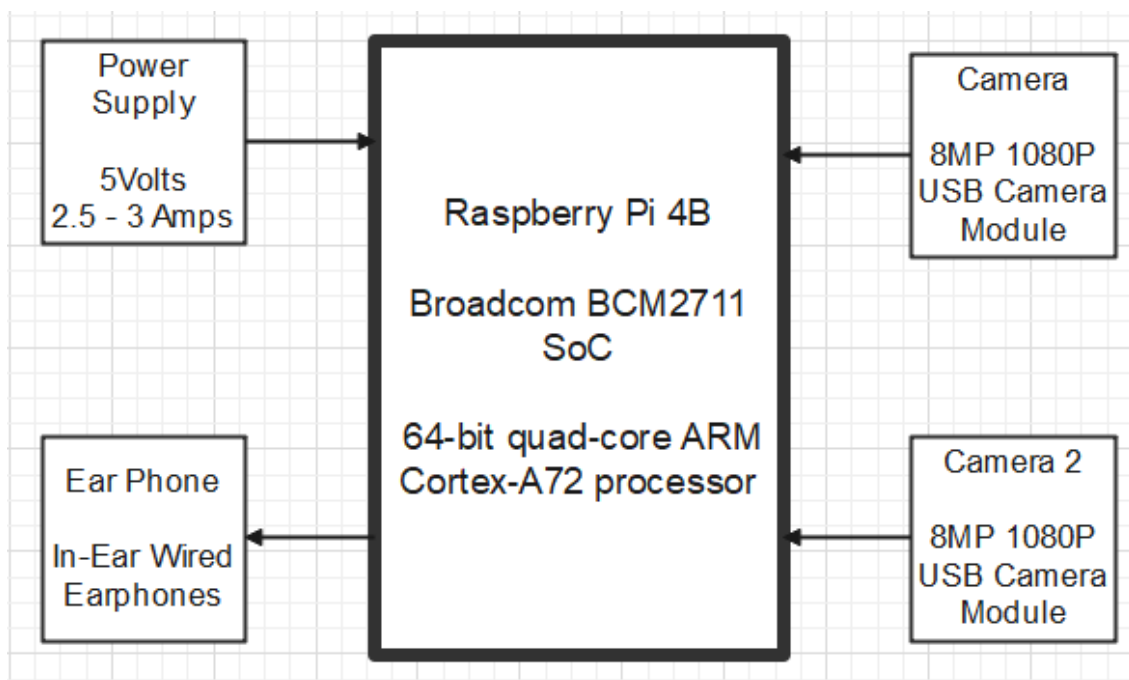


Figure 3. 1 Block diagram of the proposed system

The suggested system's complexity is depicted in the block diagram in Figure 3.1. The system is controlled by a python command run on the master Raspberry Pi 4b.

3.2.2 Headphones

The headphones will be used to help the user listen to directions given by the raspberry pi. The headphones are going to be small, lightweight and connected to the glasses, so the user will not be concerned about losing them or feel uncomfortable wearing them.



Figure 3. 2 Earpiece [14]

3.2.3 Raspberry pi 4 b



Figure 3. 3 Raspberry pi 4b [15]

The raspberry pi microcontrollers as shown in fig 3.2 and 3.3 receive live video from the stereo (computer vision) camera and process it to provide a path to the user through headphones. The python program required to process the real-time video is stored on the Raspberry Pi.

3.2.4 Vibration motor

This motor as seen in Fig 3.4 will connect to the raspberry pi and vibrate to alert the user. The vibration motor will receive signal and vibrate to either side when being called upon.



Figure 3. 4 Vibration motor [16]

3.2.5 Design specifications

The specifications for the system proposed are shown below:

Table 3. 1 Raspberry pi 4b design specification

Parameter	Specification
Power supply voltage	5v
RAM	2Gb, 4Gb, 8Gb
CPU	Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz.
Operating system	Linux, installed via NOOBS.
Amperes	3Amps

Table 3. 2 USB Camera Module design specification

Parameter	Specification
Power supply voltage	5v
Resolution	8MP 3264H x 2448V
Amperes	200Ma

3.2.6 Power Draft

This summarizes the overall power used by the project

Parameter	Specification
Power supply voltage	5v
Amperes	2.5 – 3A

3.3 Software design

The various sub-components of the software system that have been used to make the smart glasses system are described below. As shown in Fig 3.5, the flowchart below gives a detailed description on work the system works.

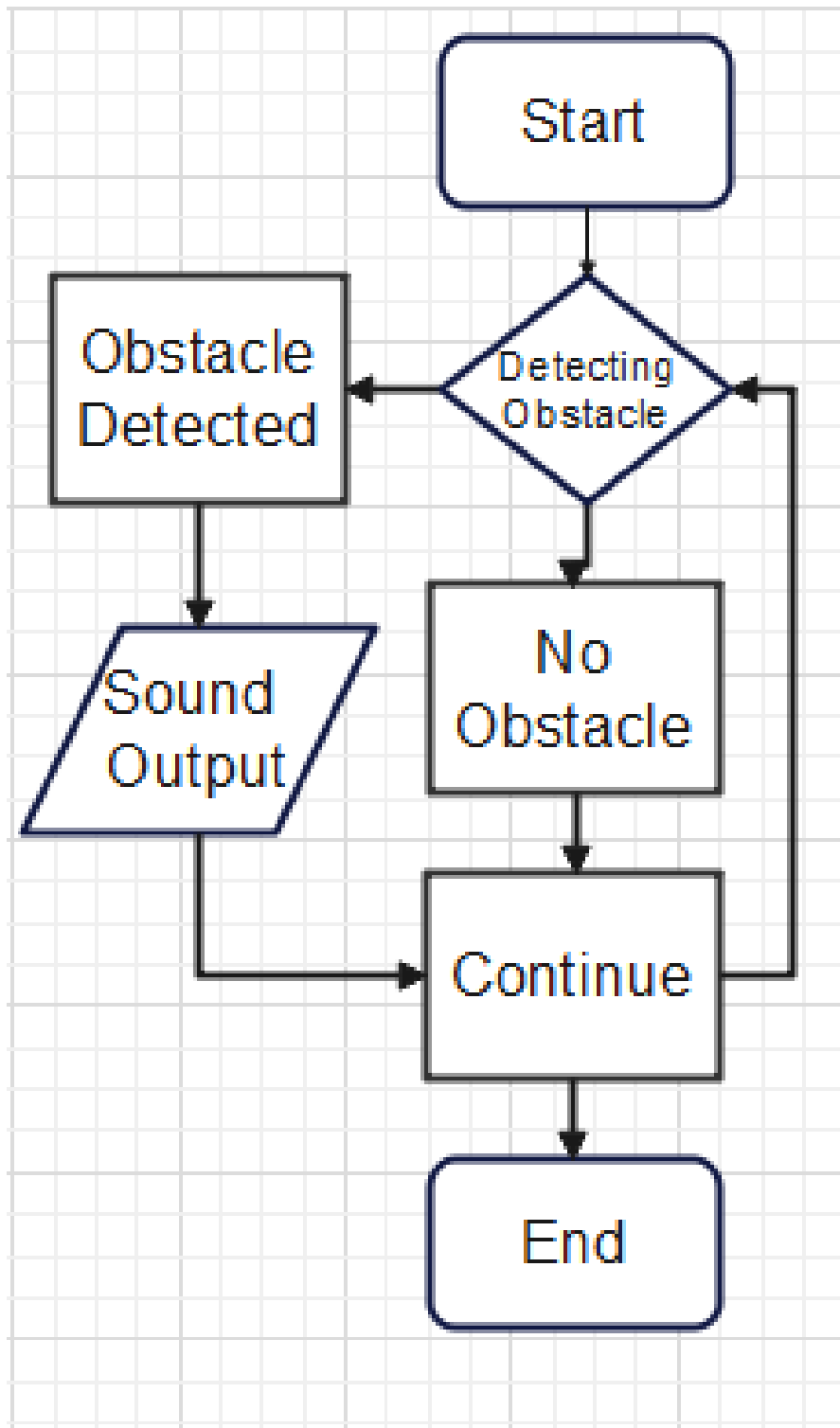


Figure 3. 5 Flowchart of system

3.3.1 Raspbian OS

In the proposed system, the device is controlled by a Raspberry Pi running the Raspbian operating system. It comes with a web browser, a

menu bar, a file manager, and plenty of desktop shortcuts for pre-installed programs. The device's performance is improved as a result, allowing it to be used in a variety of applications including numerous floating-point arithmetic operations. Other apps gain performance by exploiting the enhanced instructions of the ARMv6 CPU in the Raspberry Pi. It supports a number of programming languages, including C, C++, Java, Python, and others.

3.3.2 Python Programming

Python is one of the most widely used programming languages. Guido van Rossum created it, and it was released in 1991.

To make this project possible, python was majorly used to code the needed functions. A lot of libraries were used from python including keras, matplotlib and others.

1. OpenCV

OpenCV is a large open-source library for computer vision, machine learning, and image processing, and it currently plays an essential part in real-time operations, which are critical in today's systems. It may be used to detect items, people, and even human handwriting in photos and movies. Python can process the OpenCV array structure for analysis when it is combined with other libraries like NumPy. We employ vector space and execute mathematical operations on these characteristics to identify visual patterns and their different features. [17]

Applications of OpenCV: There are lots of applications which are solved using OpenCV, some of them are listed below

- face recognition
- Automated inspection and surveillance

- Video/image search and retrieval
- Robot and driver-less car navigation and control
- object recognition

Use in Project:

As discussed in the introduction and applications of opencv, this library was primarily used to calibrate the camera's and for object detection.

Camera Calibration:

The main purpose of this is to remove distortion.

- **Chess Board pattern**

```
cv.drawChessboardCorners(image, patternSize, corners,  
                          patternWasFound) ->image
```

The recognized chessboard corners are rendered. If the board was not discovered, the function draws individual chessboard corners as red circles; if the board was identified, the function draws colored corners joined with lines.

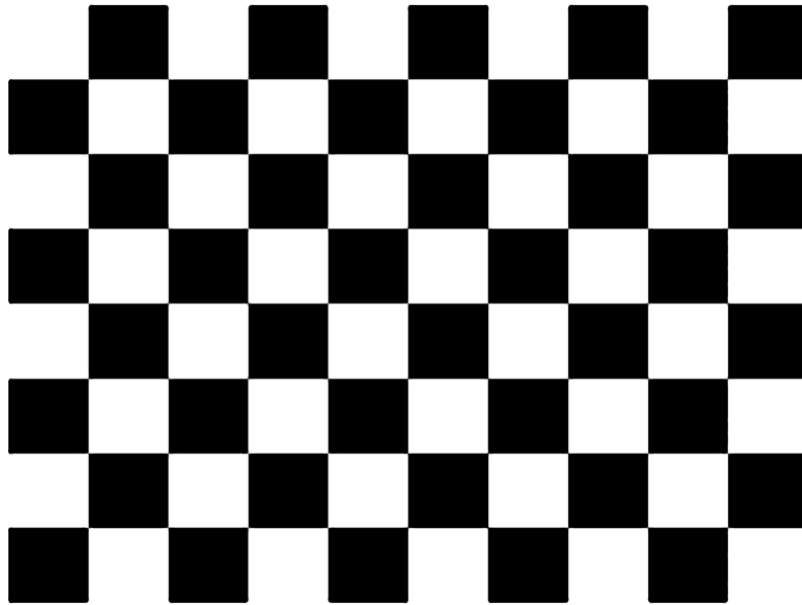


Figure 3. 6 Chess Board Pattern

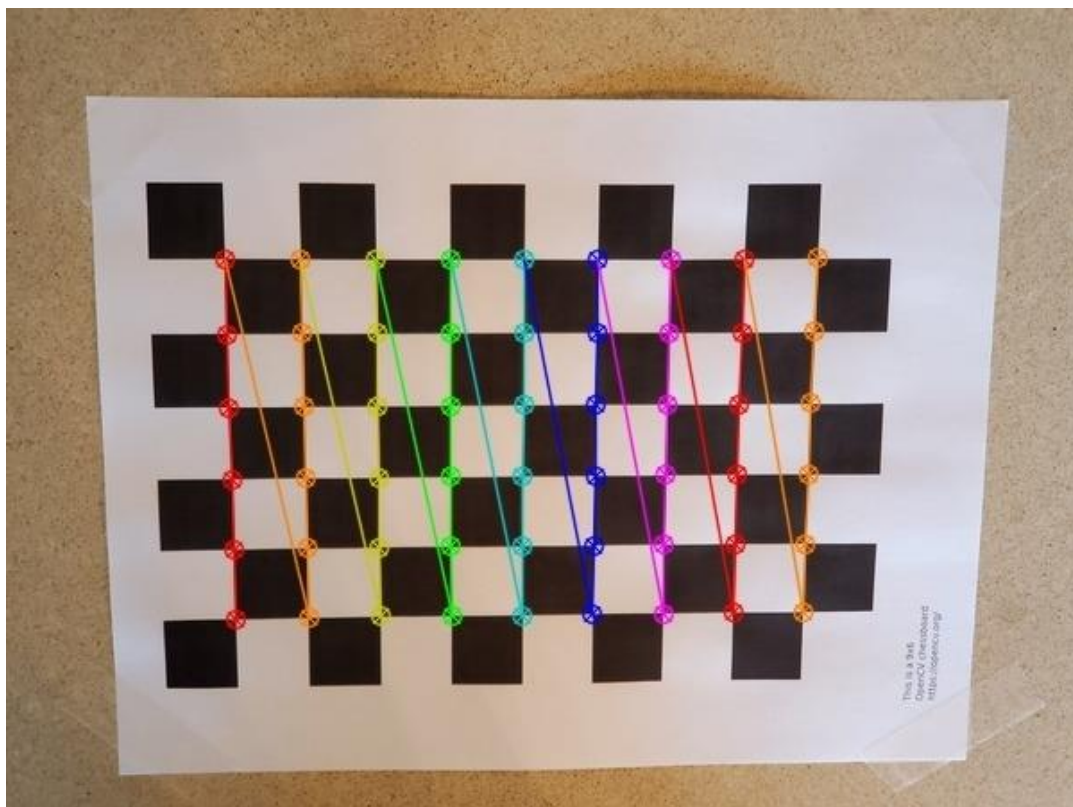


Figure 3. 7 Chess Board Pattern Identified

The patterns in Fig 3.6 are what is recognized and a red dot with some interconnecting lines are used to represent that the pattern has been identified as seen in Fig 3.7.

- **Camera Calibration**

```
cv.calibrateCamera(objectPoints, imagePoints, imageSize, cameraMatrix,  
                  distCoeffs[, rvecs[, tvecs[, flags[, criteria]]])
```

From several views of a calibration pattern, OpenCV determines the camera intrinsic and extrinsic parameters.

This calibration helps relay information regarding the cameras. This is done so the system can process the information required.

2. Calibration code

Some variables are saved once the camera has been calibrated. This information is retrieved later during the execution of the programme. The data from the two binocular (stereo) vision cameras is processed in this way.

The file that holds these values, according to the code, is called "stereoMap.xml."

This code then attempts to correct and undistort the image after obtaining the data. It returns the undistorted frames to the main script after that.

3. Triangulation code

This script calculates the depth between the user and the object. This is a process of getting the required parameters, calculating the disparity between the left and right image. Then it calculates the depth from the left and right frames of the camera.

4. Main script

First, let's go through the Python packages that were utilized in this script.

- **GTTS**

Gtts (Google Text-to-Speech) is a Python package that allows you to interact with the Google Text-to-Speech API. This is used to convert text into an audio file, such as "a human is 5 centimetres away." The blind are then played this file.

- **PYGLET**

The gtts audio for the blind was played back using this python library.

CODE EXPLANATION

This python script runs the main activity of the system. This code is majorly broken into three parts:

- Object detection
- Distance estimation
- Audio playback

In terms of object detection, the script uses a model that has previously been trained. This script employs the model to determine whether or not a known obstacle exists (cars and humans in this case). Because this system uses two cameras, detection must take place on both ends of the cameras. Threading is established as a notion that allows numerous processes to execute at the same time during code execution. The code has been separated into functions to make it easier to understand. Object detection is a function that contains all of the implementation for

detecting recognized obstacles. The code is then executed for both threads, enabling detection for both cameras.

A function called depth is used to estimate distance. For compilation, this method accepts the left and right picture frames. The object detection function is then checked to see if an object has been detected. When an object is discovered, this function sends certain arguments to the triangulation script, which calculates and returns the estimated depth.

After the depth has been computed, another function called audio is called into action. This method takes the depth, converts it to a sentence, and then converts it to audio. The audio is then played for the blind to hear.

CHAPTER FOUR

IMPLEMENTATION AND RESULT

4.1 Introduction

This chapter goes through the design of the system in great depth. It also addresses the outcomes of the design process and their eventual execution. The bill of quantities will be displayed here as well.

4.2 Implementation

The goal of the project was to detect obstructions in a blind person's route. To aid the blind, this prototype uses two cameras in the form of stereo vision (computer vision). These cameras, which are mounted on each side of the glass, capture the surroundings. This data is sent into the Raspberry Pi for computation and analysis. The obstructions computed are delivered to the user in the form of audio.

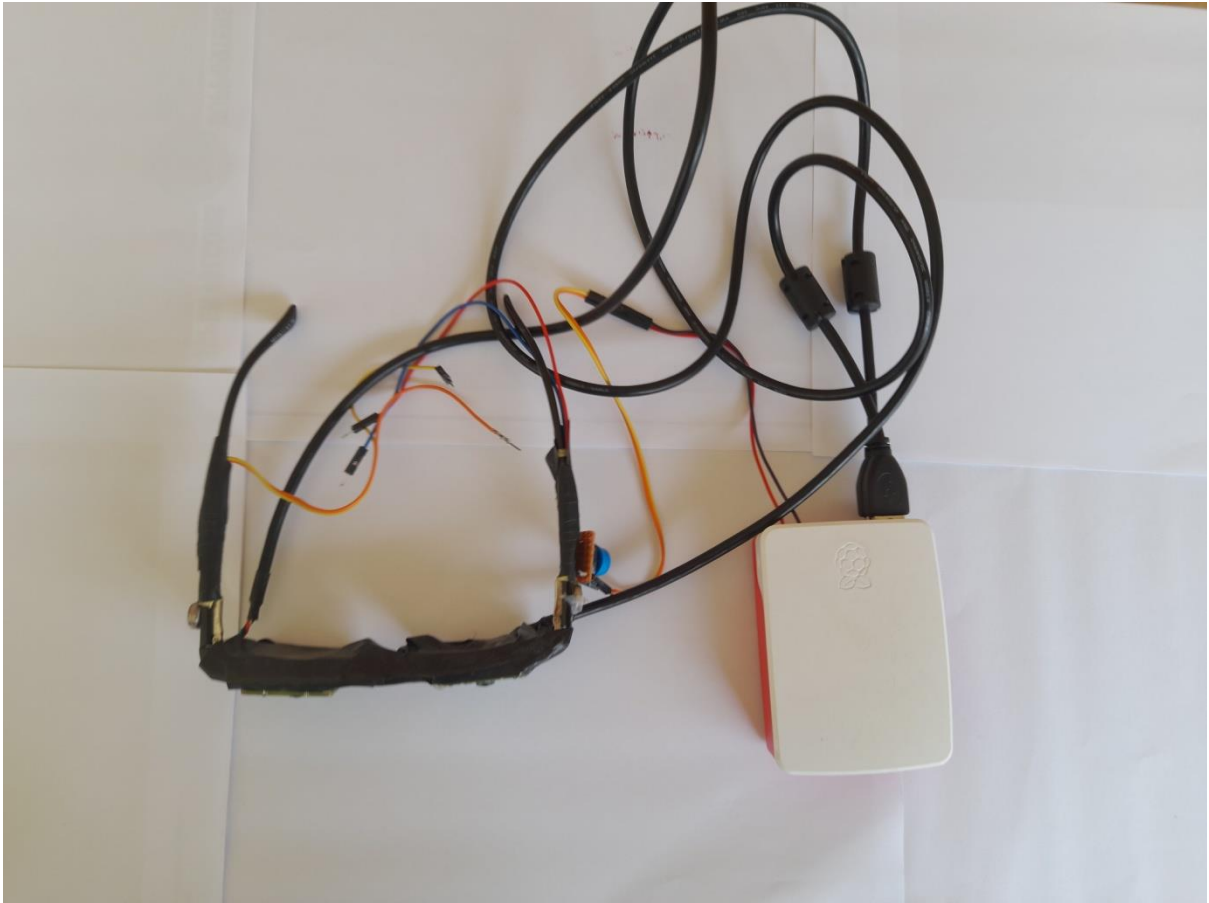


Figure 4. 1 Connection between Raspberry pi and Glasses

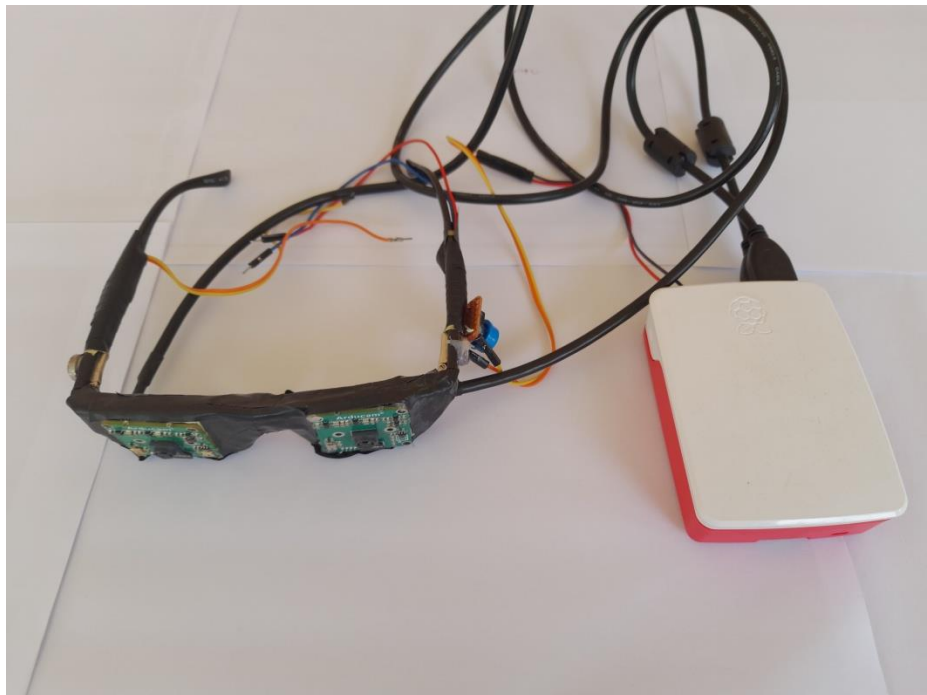


Figure 4. 2 Close-up to 8MP (USB) Camera's

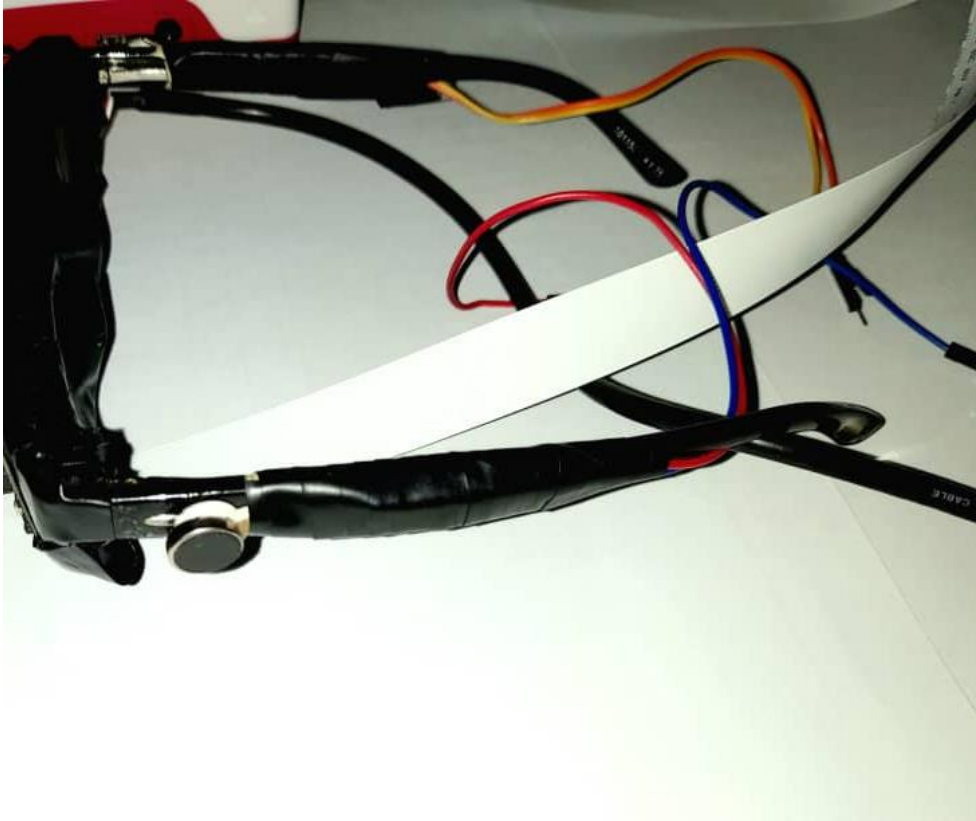


Figure 4. 3 Close-up to Vibration motor



Figure 4. 4 Glasses open to be worn by user

As displayed in Fig. 4.1, this glass was created using a Raspberry Pi and two cameras. From right to left, these two cameras (Fig 4.2), both 8MP, are connected through USB. The stereo vision system is built on this.

The Vibration motor (Fig. 4.3) is the next actuator, which vibrates depending on the distance between the user and the object.

As it is still used as a normal glass, the glass is set up to be worn as illustrated in Fig 4.4.

4.3 Mode of Operation

This prototype's design has two cameras that relay data to the Raspberry Pi. The results are then calculated using this data. The processes for getting this prototype to function will be detailed.

The two cameras installed on each side of the frame are the initial feature of this wearable glass prototype. The distance between the cameras is around 4cm, which should be enough. The cameras then send live footage to the Raspberry Pi, which uses it to do calculations.

To determine the triangulation between these two planes, the Raspberry considers the distance between the two cameras (cameras).

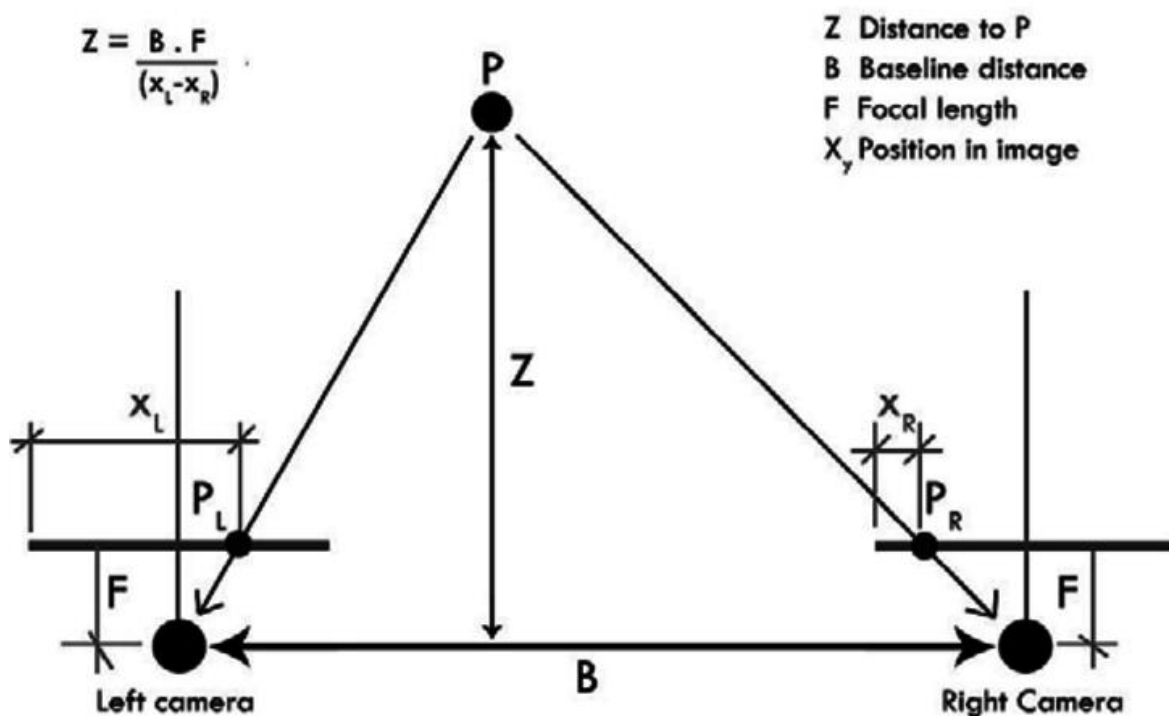


Figure 4. 5 Triangulation Stereo Vision [18]

Using the two cameras as input, Fig. 4.1 explains how triangulation works. However, the item must first be spotted before triangulation can

begin. We receive our bounding box (a virtual box created around a recognized item) using the model trained for object detection, and we have the object for triangulation to take place. The object's origin point (center point) is then determined, yielding the object's exact position. The coordinates obtained from these two cameras are then employed in the calculation

$$Z = B.F / (XL - XR)$$

The formula above calculates the distance between the user and the object.

4.4 Results

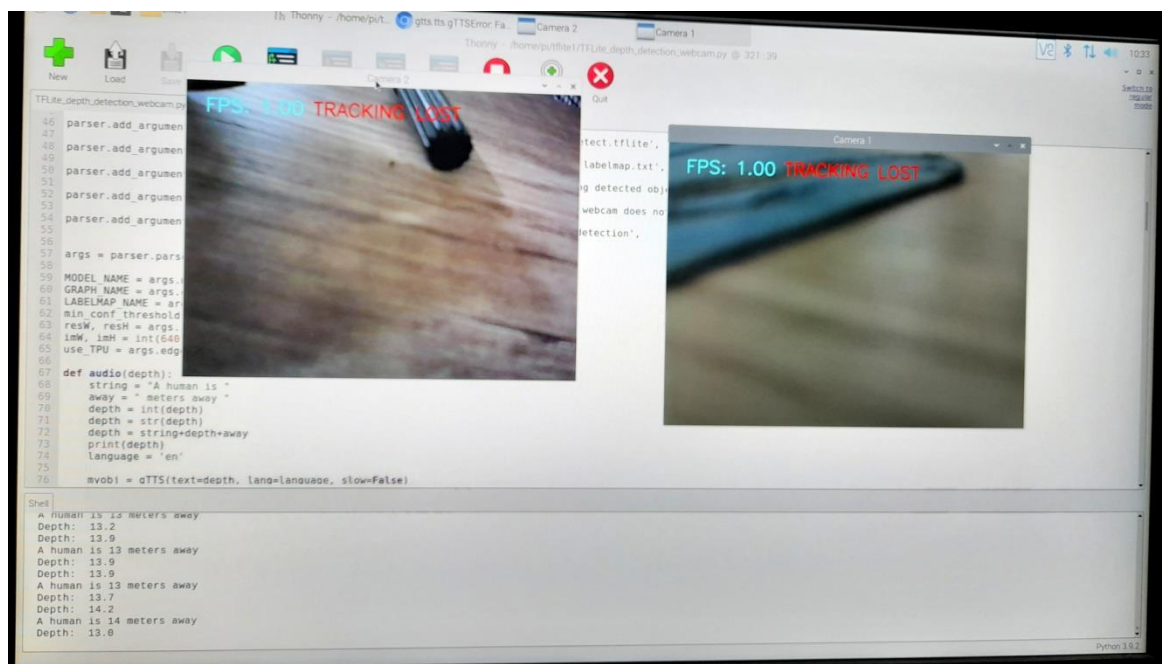


Figure 4. 6 Camera threads running

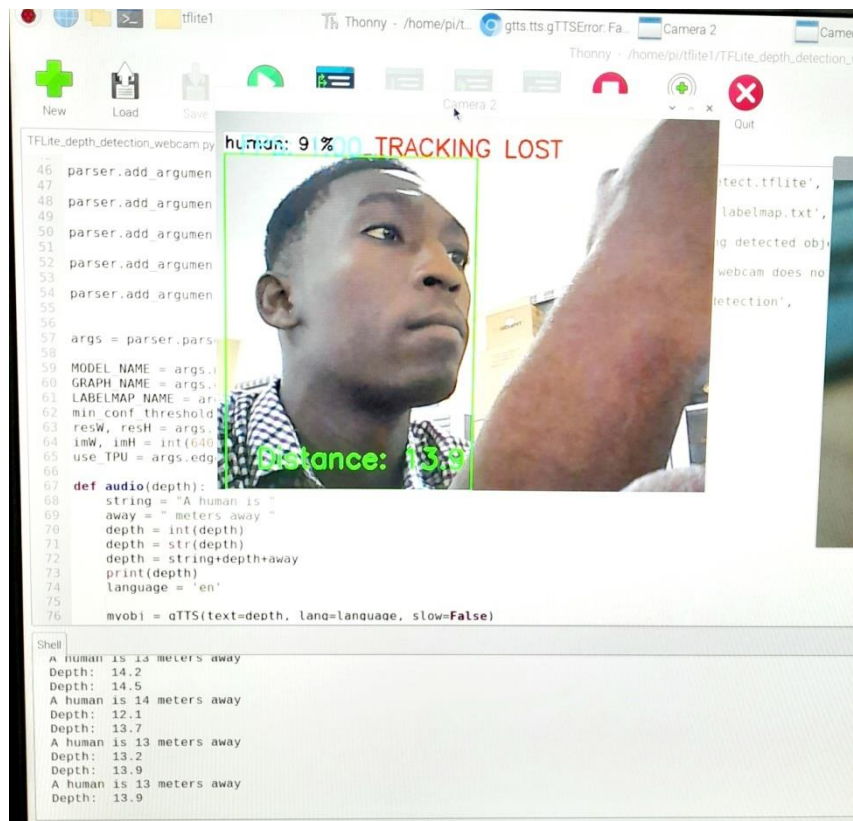


Figure 4. 7 Object detected

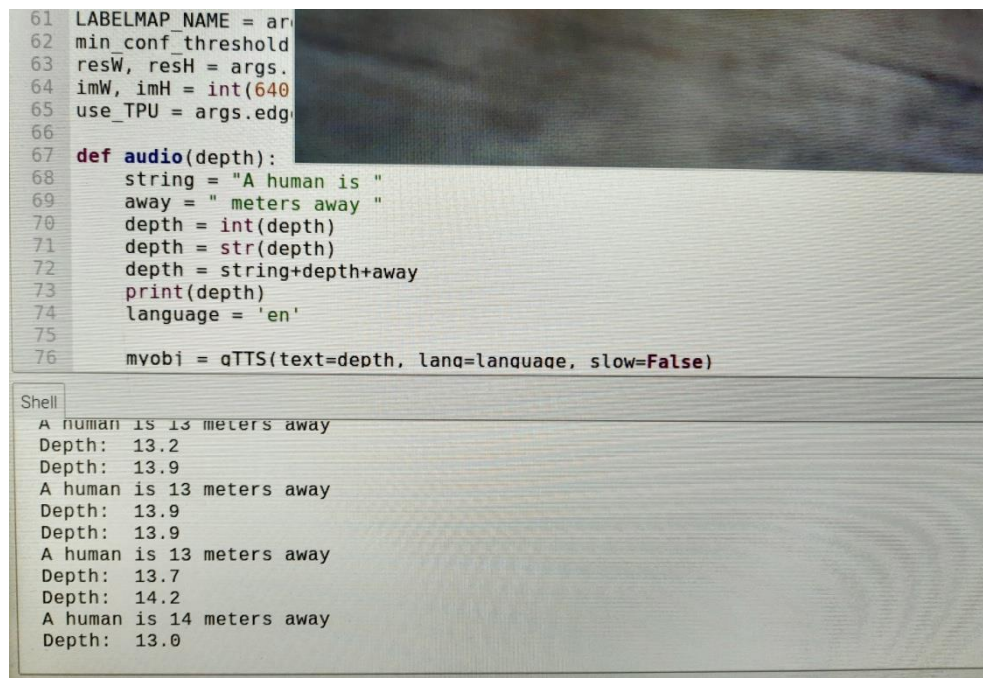


Figure 4. 8 GTTS audio function running and depth printed

The results indicate that the glasses have been fully implemented. The two cameras are shown as threads in Figure 4.2. As previously stated, this is done to guarantee that both detect objects.

When the camera identifies an item, as shown in Fig 4.3, it estimates the distance between the user and the object. This distance is measured in cm, as depicted in Figure. 4.4. This then plays an audio file for the blind person to hear and respond to.

4.5 Bill of Engineering Measurements And Evaluation

Table 4. 1 BILL OF ENGINEERING MEASUREMENTS AND EVALUATION

S/No	Components	Quantity	Unit Price (N)	Total Price (N)
1	Raspberry Pi – 2gb	1	N 34,000	N34,000
3	8MP USB Camera	2	N 17,000	N 34,000
4	Vero Board	1	N 250	N 250
5	4- piece raspberry pi heat sink	1	N 800	N 800
6	Raspberry pi case	1	N 2500	N 2500
7	Lipo 18650 Battery	2	1500	3000
8	18650 1S Lipo battery holder	2	300	600
9	Miscellaneous (shipping and others)			N22,000
	TOTAL			N 97,150

The total cost of the items provided to the school when the project is finished is shown in Table 4.1 bill of materials.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

The wearable smart glass was completely designed and implemented using Raspberry Pi. The frame was comfortable on the nose, although not as much as regular glasses.

The test revealed that the Raspberry Pi is not the most compatible hardware for completing this project. The camera's frame rate is too low to be considered dependable, with a range of 0.5 to 4.5. This is just too sluggish, especially when assisting a blind person in determining what to expect.

5.2 Challenges Encountered

This section outlines all of the work's difficulties.

- Creating an appropriate power supply was a challenge since the current had to be scaled down to 1amp, which was insufficient for the Raspberry Pi.
- Many software applications could not be run on the Raspberry Pi.
- Because of the aforementioned problem, training an appropriate object detection model proved challenging.
- Stereo (Binocular) Vision proved challenging to implement for a multitude of reasons:
 - The two cameras were originally on separate boards
 - A Raspberry Pi board does not have two camera slots
 - The two cameras must run at the same time
 - Getting data from both cameras was challenging

- The USB camera had to be detached during testing before the Pi camera slot would operate
- The Pi camera was heating up quickly
- The SD card holder broke, causing the raspberry pi to fail during usage

5.3 Recommendation

For future work and research purposes to improve on this project,

- ❖ Adding a GPS module to track the whereabouts of blind people.
- ❖ For quicker processing speed and FPS rate, use a better board or the Google Coral Dev board TPU.
- ❖ Using current code to implement path finding to lead a blind person through obstacles.
- ❖ Using the appropriate algorithms to guide a user from one map place to another.

5.4 Summary

Future technologies could be created on this prototype to aid in the integration of the blind into society. Though the system is only moderately reliable, it demonstrates what may be done to improve it. This prototype incorporates the different potentials that can be realized to assist blind people.

REFERENCES

- [1] WHO, "Blindness and vision impairment," 2021.
- [2] M. V. S. Gudlavalleti *et al.*, "Prevalence of Blindness and Visual Impairment in Nigeria: The National Blindness and Visual Impairment Survey," May 2009 2009.
- [3] medicalnewstoday, Blind person walking [image], accessed 11 may 2022,<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRGctLmYgH-UHyRu4u89phzjZq-SxCa1oy5mg&usqp=CAU>
- [4] WeCapable, "Daily Life Problems, Struggle and Challenges Faced by Blind People."
- [5] IBM, "What is computer vision?"
- [6] Thepihut, Camera Board for Raspberry Pi - Adjustable-Focus Lens (5MP) [image], accessed 21 January 2022, https://cdn.shopify.com/s/files/1/0176/3274/products/camera-board-for-raspberry-pi-adjustable-focus-lens-5mp-waveshare-wav-8193-30241755693251_500x.jpg?v=1646620749
- [7] J. C. Read, "Stereo vision and strabismus," *Eye*, vol. 29, no. 2, pp. 214-224, 2015.
- [8] Mygreatlearning, Object Detection [image], accessed 11 may 2022,<https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTDdMrU0NGskffDnoXh5RF1NwTqMnfHESd2lQ&usqp=CAU>
- [9] H. A. Said, L. Alkhatib, A. Aloraidh, and S. Alhaidar, "Smart Glasses for Blind people," 2018.
- [10] R. Agarwal *et al.*, "Low cost ultrasonic smart glasses for blind," in *2017 8th IEEE Annual Information Technology, Electronics and*

- Mobile Communication Conference (IEMCON)*, 2017: IEEE, pp. 210-213.
- [11] A. Krishnan, G. Deepakraj, N. Nishanth, and K. Anandkumar, "Autonomous walking stick for the blind using echolocation and image processing," in *2016 2nd International Conference on Contemporary Computing and Informatics (IC3I)*, 2016: IEEE, pp. 13-16.
- [12] J.-H. Kim, S.-K. Kim, T.-M. Lee, Y.-J. Lim, and J. Lim, "Smart Glasses using Deep Learning and Stereo Camera," in *2019 IEEE 8th Global Conference on Consumer Electronics (GCCE)*, 2019: IEEE, pp. 294-295.
- [13] M. N. Abdullah and A. E. Korial, "Planning the path and avoidance obstacles for visually impaired/blind people," *IOSR Journal of Computer Engineering*, vol. 17, pp. 147-151, 2015.
- [14] ubuy, Earpiece [image], accessed 11 may 2022, <https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTCT29LE6ckF82GS8FL5HfZLnE1akFBeyHLFQ&usqp=CAU>
- [15] Thepihut, Raspberry Pi 4 Model B[image], accessed 21 January 2022, https://cdn.shopify.com/s/files/1/0176/3274/products/raspberry-pi-4-model-b-raspberry-pi-sc0193-22798931034307_500x.jpg?v=1646247440
- [16] Hub360, Vibration-motor [image], accessed 11 may 2022, <https://hub360.com.ng/wp-content/uploads/2017/05/1-2.jpg>
- [17] GeekforGeek, "OpenCV - Overview," August 2021.
- [18] Researchgate, Stereo Triangulation [image], accessed 7 May 2022, <https://www.researchgate.net/publication/305110045/figure/fig1/AS:382299624493056@1468158741156/Stereo-Triangulation.png>

APPENDIX

Main Script

```
import cv2

import threading

import os

import argparse

import cv2

import numpy as np

import sys

import time

import pygame

import importlib.util

from gpiozero import Button

from gtts import gTTS


import triangulation as tri

import calibration


btn = Button(24)    # GPIO 24 (pin # 18)


class camThread(threading.Thread):
```

```

def __init__(self, previewName, camID):

    threading.Thread.__init__(self)

    self.previewName = previewName

    self.camID = camID

def run(self):

    print ("Starting " + self.previewName)

    camPreview(self.previewName, self.camID)

```

```

B = 4          #Distance between the cameras [cm]

```

```

f = 6          #Camera lense's focal length [mm]

```

```

alpha = 56.6

```

```

detect = False

```

```

detect_right = False

```

```

detect_left = False

```

```

center_point_right = [0,0]

```

```

center_point_left = [0,0]

```

```

# Define and parse input arguments

```

```

parser = argparse.ArgumentParser()

```

```

parser.add_argument('--modeldir', help='Folder the .tflite file is located in',
                    required=True)

```

```

parser.add_argument('--graph', help='Name of the .tflite file, if different than detect.tflite',
                    default='detect.tflite')

parser.add_argument('--labels', help='Name of the labelmap file, if different than
labelmap.txt',
                    default='labelmap.txt')

parser.add_argument('--threshold', help='Minimum confidence threshold for displaying
detected objects',
                    default=0.7)

parser.add_argument('--resolution', help='Desired webcam resolution in WxH. If the webcam
does not support the resolution entered, errors may occur.',
                    default='1280x720')

parser.add_argument('--edgetpu', help='Use Coral Edge TPU Accelerator to speed up
detection',
                    action='store_true')

args = parser.parse_args()

MODEL_NAME = args.modeldir

GRAPH_NAME = args.graph

LABELMAP_NAME = args.labels

min_conf_threshold = float(args.threshold)

resW, resH = args.resolution.split('x')

imW, imH = int(640), int(480)

use_TPU = args.edgetpu

def audio(depth):

```

```

string = "A human is "
away = " meters away "

depth = int(depth)
depth = str(depth)

depth = string+depth+away

print(depth)

language = 'en'

myobj = gTTS(text=depth, lang=language, slow=False)

myobj.save("depth.mp3")

music = pygamelet.resource.media("depth.mp3")
music.play()

```

```

# Import TensorFlow libraries

```

```

# If tfLite_runtime is installed, import interpreter from tfLite_runtime, else import from regular
tensorflow

```

```

# If using Coral Edge TPU, import the load_delegate library

```

```

def depth(frame,previewName):

```

```

    global frame_right

```

```

    global frame_left

```

```

    frame_right, frame_left = calibration.undistortRectify(frame_right, frame_left)

```



```

if detect == True:

    #..... CALCULATING DEPTH .....

    depth = tri.find_depth(center_point_right, center_point_left, frame_right, frame_left, B, f,
alpha)

    if previewName == "Camera 1":

        cv2.putText(frame, "Distance: " + str(round(depth,1)), (50,450),
cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0,255,0),3)

    if previewName == "Camera 2":

        cv2.putText(frame, "Distance: " + str(round(depth,1)), (50,450),
cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0,255,0),3)

    # Multiply computer value with 205.8 to get real-life depth in [cm]. The factor was found
manually.

    print("Depth: ", str(round(depth,1)))

if os.path.exists("depth.mp3"):

    os.remove("depth.mp3")

    #start_time = threading.Timer(300000,audio(depth))

else:

    audio(depth)

def objectDetection(frame,previewName):

    global detect

    global detect_right

    global detect_left

```

```

global center_point_right

global center_point_left


global GRAPH_NAME


pkg = importlib.util.find_spec('tflite_runtime')

if pkg:

    from tflite_runtime.interpreter import Interpreter

    if use_TPU:

        from tflite_runtime.interpreter import load_delegate

    else:

        from tensorflow.lite.python.interpreter import Interpreter

        if use_TPU:

            from tensorflow.lite.python.interpreter import load_delegate


# If using Edge TPU, assign filename for Edge TPU model

if use_TPU:

    # If user has specified the name of the .tflite file, use that name, otherwise use default
    'edgetpu.tflite'

    if (GRAPH_NAME == 'detect.tflite'):

        GRAPH_NAME = 'edgetpu.tflite'


# Get path to current working directory

```

```

CWD_PATH = os.getcwd()

# Path to .tflite file, which contains the model that is used for object detection

PATH_TO_CKPT = os.path.join(CWD_PATH,MODEL_NAME,GRAPH_NAME)

# Path to label map file

PATH_TO_LABELS = os.path.join(CWD_PATH,MODEL_NAME,LABELMAP_NAME)

# Load the label map

with open(PATH_TO_LABELS, 'r') as f:

    labels = [line.strip() for line in f.readlines()]

# Have to do a weird fix for label map if using the COCO "starter model" from
# https://www.tensorflow.org/lite/models/object_detection/overview
# First label is '???' , which has to be removed.

if labels[0] == '???':

    del(labels[0])

# Load the Tensorflow Lite model.

# If using Edge TPU, use special load_delegate argument

if use_TPU:

    interpreter = Interpreter(model_path=PATH_TO_CKPT,

                             experimental_delegates=[load_delegate('libedgetpu.so.1.0')])

    print(PATH_TO_CKPT)

```

```

else:

    interpreter = Interpreter(model_path=PATH_TO_CKPT)

    interpreter.allocate_tensors()

    # Get model details

    input_details = interpreter.get_input_details()
    output_details = interpreter.get_output_details()

    height = input_details[0]['shape'][1]

    width = input_details[0]['shape'][2]

    floating_model = (input_details[0]['dtype'] == np.float32)

    input_mean = 127.5

    input_std = 127.5

    # Check output layer name to determine if this model was created with TF2 or TF1,
    # because outputs are ordered differently for TF2 and TF1 models

    outname = output_details[0]['name']

    if ('StatefulPartitionedCall' in outname): # This is a TF2 model

        boxes_idx, classes_idx, scores_idx = 1, 3, 0

    else: # This is a TF1 model

        boxes_idx, classes_idx, scores_idx = 0, 1, 2

```

```

# Initialize frame rate calculation

frame_rate_calc = 1

freq = cv2.getTickFrequency()

#.....

    # Start timer (for calculating frame rate)

t1 = cv2.getTickCount()


#cv2.imshow(previewName, frame)


frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

frame_resized = cv2.resize(frame_rgb, (width, height))

input_data = np.expand_dims(frame_resized, axis=0)


# Normalize pixel values if using a floating model (i.e. if model is non-quantized)

if floating_model:

    input_data = (np.float32(input_data) - input_mean) / input_std


# Perform the actual detection by running the model with the image as input

interpreter.set_tensor(input_details[0]['index'],input_data)

interpreter.invoke()


# Retrieve detection results

```

```
boxes = interpreter.get_tensor(output_details[boxes_idx]['index'])[0] # Bounding box
coordinates of detected objects
```

```
classes = interpreter.get_tensor(output_details[classes_idx]['index'])[0] # Class index of
detected objects
```

```
scores = interpreter.get_tensor(output_details[scores_idx]['index'])[0] # Confidence of
detected objects
```

```
# Loop over all detections and draw detection box if confidence is above minimum
threshold
```

```
if previewName == "Camera 1":
```

```
    for i in range(len(scores)):
```

```
        if ((scores[i] > min_conf_threshold) and (scores[i] <= 1.0)):
```

```
            # Get bounding box coordinates and draw box
```

```
            # Interpreter can return coordinates that are outside of image dimensions, need to
            force them to be within image using max() and min()
```

```
            ymin = int(max(1,(boxes[i][0] * imH)))
```

```
            xmin = int(max(1,(boxes[i][1] * imW)))
```

```
            ymax = int(min(imH,(boxes[i][2] * imH)))
```

```
            xmax = int(min(imW,(boxes[i][3] * imW)))
```

```
            cv2.rectangle(frame, (xmin,ymin), (xmax,ymax), (10, 255, 0), 2)
```

```
            # Draw label
```

```

        object_name = labels[int(classes[i])] # Look up object name from "labels" array
        using class index

        label = '%s: %d%%' % (object_name, int(scores[i]*100)) # Example: 'person: 72%'

        labelSize, baseLine = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX,
        0.7, 2) # Get font size

        label_ymin = max(ymin, labelSize[1] + 10) # Make sure not to draw label too close
        to top of window

        cv2.rectangle(frame, (xmin, label_ymin-labelSize[1]-10), (xmin+labelSize[0],
        label_ymin+baseLine-10), (255, 255, 255), cv2.FILLED) # Draw white box to put label text in

        cv2.putText(frame, label, (xmin, label_ymin-7), cv2.FONT_HERSHEY_SIMPLEX,
        0.7, (0, 0, 0), 2) # Draw label text

        center_point_right = ((xmin + xmax)/ 2, (ymin + ymax) / 2)

        detect = True

        depth(frame,previewName)

    else:

        cv2.putText(frame, "TRACKING LOST", (200,50),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255),2)

    if previewName == "Camera 2":

        for i in range(len(scores)):

            if ((scores[i] > min_conf_threshold) and (scores[i] <= 1.0)):

                # Get bounding box coordinates and draw box

                # Interpreter can return coordinates that are outside of image dimensions, need to
                force them to be within image using max() and min()

                ymin = int(max(1,(boxes[i][0] * imH)))

                xmin = int(max(1,(boxes[i][1] * imW)))

                ymax = int(min(imH,(boxes[i][2] * imH)))

```

```

xmax = int(min(imW,(boxes[i][3] * imW)))

cv2.rectangle(frame, (xmin,ymin), (xmax,ymax), (10, 255, 0), 2)

# Draw label

object_name = labels[int(classes[i])] # Look up object name from "labels" array
using class index

label = '%s: %d%%' % (object_name, int(scores[i]*100)) # Example: 'person: 72%'

labelSize, baseLine = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX,
0.7, 2) # Get font size

label_ymin = max(ymin, labelSize[1] + 10) # Make sure not to draw label too close
to top of window

cv2.rectangle(frame, (xmin, label_ymin-labelSize[1]-10), (xmin+labelSize[0],
label_ymin+baseLine-10), (255, 255, 255), cv2.FILLED) # Draw white box to put label text in

cv2.putText(frame, label, (xmin, label_ymin-7), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 0, 0), 2) # Draw label text

center_point_left = ((xmin + xmax)/ 2, (ymin + ymax) / 2)

detect = True

depth(frame,previewName)

else:

cv2.putText(frame, "TRACKING LOST", (200,50),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255),2)

```



```

#global frame_rate_calc

# Draw framerate in corner of frame

cv2.putText(frame,'FPS:
{0:.2f}'.format(frame_rate_calc),(30,50),cv2.FONT_HERSHEY_SIMPLEX,1,(255,255,0),2,cv
2.LINE_AA)

# All the results have been drawn on the frame, so it's time to display it.

#cv2.imshow('Object detector', frame)

cv2.imshow(previewName, frame)


# Calculate framerate

t2 = cv2.getTickCount()

time1 = (t2-t1)/freq

frame_rate_calc= 1/time1


def camPreview(previewName, camID):

    global frame_right

    global frame_left

    cv2.namedWindow(previewName)

    cam = cv2.VideoCapture(camID)

    ret = cam.set(3,imW)

    ret = cam.set(4,imH)

    if cam.isOpened(): # try to get the first frame

        rval, frame = cam.read()

```

else:

 rval = False

if previewName == "Camera 1":

 frame_right = frame

if previewName == "Camera 2":

 frame_left = frame

while rval:

 rval, frame = cam.read()

 objectDetection(frame,previewName)

 # Press 'q' to quit

 if cv2.waitKey(1) == ord('q'):

 break

cv2.destroyAllWindows(previewName)

try:

if os.path.exists("depth.mp3"):

 os.remove("depth.mp3")

print('Waiting for button...')

btn.wait_for_press()

```
# Create two threads as follows
```

```
thread1 = camThread("Camera 1", 0)
```

```
thread2 = camThread("Camera 2", 1)
```

```
thread1.start()
```

```
thread2.start()
```

```
except (Exception, KeyboardInterrupt) as e:
```

```
    print('\n>> Process Exception {}'.format(e))
```

Triangulation code

```
import sys

import cv2

import numpy as np

import time

def find_depth(right_point, left_point, frame_right, frame_left, baseline,f, alpha):

    # CONVERT FOCAL LENGTH f FROM [mm] TO [pixel]:

    height_right, width_right, depth_right = frame_right.shape

    height_left, width_left, depth_left = frame_left.shape


    if width_right == width_left:

        f_pixel = (width_right * 0.5) / np.tan(alpha * 0.5 * np.pi/180)


    else:

        print('Left and right camera frames do not have the same pixel width')

    x_right = right_point[0]

    x_left = left_point[0]


    # CALCULATE THE DISPARITY:

    disparity = x_left-x_right    #Displacement between left and right frames [pixels]

    # CALCULATE DEPTH z:

    zDepth = (baseline*f_pixel)/disparity    #Depth in [cm]


    return zDepth
```

Calibration code

```
import sys

import numpy as np

import time

import imutils

import cv2

# Camera parameters to undistort and rectify images

cv_file = cv2.FileStorage()

cv_file.open('stereoMap.xml', cv2.FileStorage_READ)

stereoMapL_x = cv_file.getNode('stereoMapL_x').mat()

stereoMapL_y = cv_file.getNode('stereoMapL_y').mat()

stereoMapR_x = cv_file.getNode('stereoMapR_x').mat()

stereoMapR_y = cv_file.getNode('stereoMapR_y').mat()

def undistortRectify(frameR, frameL):

    # Undistort and rectify images

    undistortedL= cv2.remap(frameL, stereoMapL_x, stereoMapL_y, cv2.INTER_LANCZOS4,
cv2.BORDER_CONSTANT, 0)

    undistortedR=      cv2.remap(frameR,      stereoMapR_x,      stereoMapR_y,
cv2.INTER_LANCZOS4, cv2.BORDER_CONSTANT, 0)

    return undistortedR, undistortedL
```

Stereo_calibration code

```
import numpy as np

import cv2 as cv

import glob

from matplotlib import pyplot as plt

##### FIND CHESSBOARD CORNERS - OBJECT POINTS AND IMAGE
POINTS #####

chessboardSize = (9,6)

frameSize = (640,480)

# termination criteria

criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....., (6,5,0)

objp = np.zeros((chessboardSize[0] * chessboardSize[1], 3), np.float32)

objp[:, :2] = np.mgrid[0:chessboardSize[0], 0:chessboardSize[1]].T.reshape(-1, 2)

objp = objp * 20

#print(objp)

# Arrays to store object points and image points from all the images.

objpoints = [] # 3d point in real world space

imgpointsL = [] # 2d points in image plane.

imgpointsR = [] # 2d points in image plane.
```

```

imagesLeft = sorted(glob.glob('C:/Users/rotim/Documents/STEREO_VISION/L/*.jpg'))
imagesRight = sorted(glob.glob('C:/Users/rotim/Documents/STEREO_VISION/R/*.jpg'))

for imgLeft, imgRight in zip(imagesLeft, imagesRight):

    imgL = cv.imread(imgLeft)

    imgR = cv.imread(imgRight)

    grayL = cv.cvtColor(imgL, cv.COLOR_BGR2GRAY)
    grayR = cv.cvtColor(imgR, cv.COLOR_BGR2GRAY)

    # Find the chess board corners

    retL, cornersL = cv.findChessboardCorners(grayL, chessboardSize, None)
    retR, cornersR = cv.findChessboardCorners(grayR, chessboardSize, None)

    # If found, add object points, image points (after refining them)

    if retL and retR == True:

        objpoints.append(objp)

        cornersL = cv.cornerSubPix(grayL, cornersL, (11,11), (-1,-1), criteria)
        imgpointsL.append(cornersL)

        cornersR = cv.cornerSubPix(grayR, cornersR, (11,11), (-1,-1), criteria)
        imgpointsR.append(cornersR)

```

```

# Draw and display the corners

cv.drawChessboardCorners(imgL, chessboardSize, cornersL, retL)

cv.imshow('img left', imgL)

cv.drawChessboardCorners(imgR, chessboardSize, cornersR, retR)

cv.imshow('img right', imgR)

cv.waitKey(500)

cv.destroyAllWindows()

##### CALIBRATION #####

retL, cameraMatrixL, distL, rvecsL, tvecsL = cv.calibrateCamera(objpoints, imgpointsL,
frameSize, None, None)

heightL, widthL, channelsL = imgL.shape

newCameraMatrixL, roi_L = cv.getOptimalNewCameraMatrix(cameraMatrixL, distL, (widthL,
heightL), 1, (widthL, heightL))

retR, cameraMatrixR, distR, rvecsR, tvecsR = cv.calibrateCamera(objpoints, imgpointsR,
frameSize, None, None)

heightR, widthR, channelsR = imgR.shape

newCameraMatrixR, roi_R = cv.getOptimalNewCameraMatrix(cameraMatrixR, distR,
(widthR, heightR), 1, (widthR, heightR))

print(cameraMatrixL)

print(newCameraMatrixL)

```



```
##### Stereo Vision Calibration #####
```

```
flags = 0
```

```
flags |= cv.CALIB_FIX_INTRINSIC
```

```
# Here we fix the intrinsic camera matrixes so that only Rot, Trns, Emat and Fmat are  
calculated.
```

```
# Hence intrinsic parameters are the same
```

```
criteria_stereo = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)
```

```
# This step is performed to transformation between the two cameras and calculate Essential  
and Fundamental matrix
```

```
retStereo, newCameraMatrixL, distL, newCameraMatrixR, distR, rot, trans, essentialMatrix,  
fundamentalMatrix = cv.stereoCalibrate(objpoints, imgpointsL, imgpointsR,  
newCameraMatrixL, distL, newCameraMatrixR, distR, grayL.shape[:-1], criteria_stereo,  
flags)
```

```
# Reprojection Error
```

```
mean_error = 0
```

```
for i in range(len(objpoints)):
```

```
    imgpoints2, _ = cv.projectPoints(objpoints[i], rvecsL[i], tvecsL[i], newCameraMatrixL,  
    distL)
```

```
    error = cv.norm(imgpointsL[i], imgpoints2, cv.NORM_L2)/len(imgpoints2)
```

```
    mean_error += error
```

```
print("total error: {}".format(mean_error/len(objpoints)))
```

```
##### Stereo Rectification #####
```

```
rectifyScale= 1
```

```
rectL, rectR, projMatrixL, projMatrixR, Q, roi_L, roi_R= cv.stereoRectify(newCameraMatrixL,  
distL, newCameraMatrixR, distR, grayL.shape[:-1], rot, trans, rectifyScale,(0,0))
```

```
print(Q)
```

```
stereoMapL = cv.initUndistortRectifyMap(newCameraMatrixL, distL, rectL, projMatrixL,  
grayL.shape[:-1], cv.CV_16SC2)
```

```
stereoMapR = cv.initUndistortRectifyMap(newCameraMatrixR, distR, rectR, projMatrixR,  
grayR.shape[:-1], cv.CV_16SC2)
```

```
print("Saving parameters!")
```

```
cv_file = cv.FileStorage('stereoMap.xml', cv.FILE_STORAGE_WRITE)
```

```
cv_file.write('stereoMapL_x',stereoMapL[0])
```

```
cv_file.write('stereoMapL_y',stereoMapL[1])
```

```
cv_file.write('stereoMapR_x',stereoMapR[0])
```

```
cv_file.write('stereoMapR_y',stereoMapR[1])
```

```
cv_file.write('q', Q)
```

```
cv_file.write('cl', newCameraMatrixL)
```

```
cv_file.write('cr', newCameraMatrixR)
```

```
cv_file.write('dl', distL)
```

```
cv_file.write('dr', distR)
```

```
cv_file.release()
```